Cでmallocしてゼロクリアして、データを詰める。これをRustで書いた場合とで比較したい。特に安全性に関する理解を得たい

CとRustで「mallocしてゼロクリアしてデータを詰める」処理は、見た目が似ていても安全性の前提がかなり違います。[1] [2]

Cでの典型コードと問題点

例:可変長バッファを確保してゼロクリアし、使い終わったら解放。

```
#include <stdlib.h>
#include <string.h>

unsigned char *alloc_and_fill(size_t len) {
    unsigned char *buf = malloc(len);
    if (!buf) return NULL;

    memset(buf, 0, len);            // ゼロクリア
    // 何かデータを詰める
    // buf[i] = ...;

    return buf;                     // 呼び出し側がfreeする責任
}
```

ここで起きうる典型的な問題:

- `malloc`の戻り値チェック漏れ (NULL参照、未定義動作)。[3]
- `memset`のサイズミス (len計算バグでバッファオーバーラン)。[4]
- 呼び出し側が`free`を忘れる／二重`free`する (メモリリーク・ダングリングポインタ)。[3]
- `buf`の寿命・所有者がコメントと慣習でしか表現されない。

つまり「正しく書くことはできる」が、コンパイラはほとんど助けてくれません。

Rustで同じことを安全に書く

同等の「ヒープにlenバイト確保してゼロクリアし、そこに詰める」を安全側だけで書くと：

```
fn alloc_and_fill(len: usize) -> Vec<u8> {
    // len 個の 0 で埋めた Vec を確保（内部的にalloc_zeroedが使われる最適化あり）
    let mut buf = vec![0u8; len]; // [web:13]
```

```
    // 何かデータを詰める
    // buf[i] = ...;

    buf // 所有権ごと返す
}
```

ポイント:

- `vec![0; len]`は要素が全て０のバッファを確保する高レベルAPIで、`u8`の場合は内部で`alloc_zeroed`相当を使う特殊化が入るため、効率もよいです。[1]
- `Vec<u8>`は長さと容量を管理し、境界チェック付きの`buf[i]`や`buf.get(i)`を提供するので、範囲外アクセスをコンパイル時または実行時に検出できます。[2]
- バッファの所有権は戻り値として1か所にしか存在せず、スコープを抜けたときに自動で`free`されるため、二重解放・解放忘れが起きません。[2]

「`Box<[u8]>`で長さ固定のヒープ配列が欲しい」なら:

```
fn alloc_bytes(len: usize) -> Box<[u8]> {
    vec![0u8; len].into_boxed_slice() // [web:13]
}
```

これも同様にゼロ初期化された領域が確保されます。

RustでC並みに低レベルに書く場合

Cに近いことをわざとやると:

```
use std::alloc::{alloc_zeroed, Layout};
use std::ptr::NonNull;

fn alloc_bytes(len: usize) -> Box<[u8]> {
    let layout = Layout::array::<u8>(len).unwrap();
    let raw_ptr = unsafe { alloc_zeroed(layout) };          // 生ポインタ確保
    let ptr = NonNull::new(raw_ptr).unwrap();               // NULLチェック
    let slice_ptr = NonNull::slice_from_raw_parts(ptr, len);
    unsafe { Box::from_raw(slice_ptr.as_mut()) }            // 所有権に包む
}
```

- ここは`unsafe`ブロックが必要で、「レイアウトに合うサイズを確保する」「NULLをチェックする」「長さとポインタの対応を守る」といった前提をプログラマが守る契約になっています。[1]
- ただし一度`Box<[u8]>`にしてしまえば、以降の利用は安全側で行え、`free`相当は自動で行われます。[1]

Cと違って「危険な部分」は`unsafe`で明示され、そこから先は安全な型に隠蔽される、という構造になっています。[5] [1]

安全性の観点での比較

ざっくりとC vs Rustをテーブルにすると:

観点	C	Rust (安全なAPI使用時)
メモリ確保	`malloc/calloc`、 NULLチェック手書き	`Vec`, `Box` などが内部で確保、 エラー時はpanicなど。[1]
ゼロクリア	`memset`, `calloc` を自分で呼ぶ	`vec![0; len]` など高レベルAPI (内部で最適化)。[1]
範囲外アクセス	未定義動作、検出されないことも多い	境界チェックあり、違反時はpanicで即座に検出。[2]
ダングリングポインタ	`free`後もポインタが生き続けがち	所有権/ライフタイムでコンパイル時に禁止。[2]
メモリリーク	`free`忘れで簡単に発生	正常パスでは自動解放、 意図的リークは`mem::forget`等。[2]
非同期・複数スレッド	共有ポインタのレースコンディション	`Send`/`Sync`、借用規則で安全でない共有を禁止。[2]

「mallocしてゼロクリアして、データを詰める」というパターン自体はRustでも書けますが、Rustでは:

- 可能な限り `Vec` / `Box<[T]>` のような所有権付きコンテナを使う
- どうしても生ポインタが必要な場合でも `unsafe` に閉じ込め、外側は安全なAPIにする

という設計になるので、「安全性をコンパイラが強制できる範囲」がCよりかなり広いのが本質的な違いです。[5] [2] [1]

もしよければ、実際に使っているCのスニペットを出してもらえれば、それを1対1でRustに書き換えて、安全面の差分をコメント付きで比較します。

⁂

1. https://users.rust-lang.org/t/any-way-to-create-a-zero-filled-box-u8-without-through-vec-or-using-unsafe/109005
2. https://www.reddit.com/r/rust/comments/1lv4cln/media_rust_vs_c_assembly_can_you_see_safety_at/
3. https://kmaebashi.com/bbs/list.php?boardid=kmaebashibbs&range=800
4. https://www.youtube.com/watch?v=tWNXCX2gkIo
5. https://gist.github.com/podhmo/c2dbd881f2e6c0886a8b7c6ca1a75ada
6. https://techracho5.rssing.com/chan-22432111/all_p17.html
7. https://qiita.com/yahhoi136/items/6bcdfcae0b92dc0658b8
8. https://sozysozbot.github.io/seccamp-2022-c-compiler-seminar/
9. https://www.weblio.jp/content/C++

10. https://stackoverflow.com/questions/77136991/why-does-vecvecwith-capacityn-create-0-capacity-for-the-sub-vectors